

# Trajectory-Aware Reinforcement Learning for Discrete Prompt Compression

Thesis Defense Presentation

Muhammad Shaheer Saleh

School of Electrical Engineering and Computer Science (SEECS)  
National University of Sciences and Technology (NUST)

September 25, 2026

# Motivation: The LLM Context Bottleneck

- **Inference Cost Scaling:** Quadratic complexity in self-attention ( $\mathcal{O}(N^2)$ ) and linear KV-cache growth impose severe latency and VRAM constraints.
- **Linguistic Redundancy:** Real-world prompts contain substantial syntactic padding, boilerplate, and low-information tokens that do not contribute to downstream reasoning.
- **The Core Objective:** Learn an upstream, lightweight editor that physically prunes uninformative prompt tokens before downstream execution.
- **Core Challenge:** Preserving downstream target accuracy and generation distribution without modifying or fine-tuning massive downstream model parameters.

## Thesis Goal

Develop a discrete, trajectory-aware prompt pruning framework that enforces hard token deletion while supervising upstream policies via dense downstream causal probability divergence.

# Discrete Hard Masking vs. Continuous Soft Masking

## Soft Continuous Masks ( $w_i \in [0, 1]$ )

- Continuous attention scaling or Gumbel-Softmax relaxations.
- **Fatal Flaw:** Requires thresholding at test time to physically drop tokens.
- Induces severe **train-test distribution shift** leading to generation collapse.
- Downstream KV cache is still allocated during continuous training.

## Discrete Hard Masks ( $m_i \in \{0, 1\}$ )

- Tokens are strictly retained (1) or physically deleted (0).
- Zero train-test discrepancy: what the policy selects during training is physically passed at test time.
- **Optimization Challenge:** Non-differentiable step function prevents standard backpropagation.
- **Solution:** Supervised via policy gradient reinforcement learning (REINFORCE).

# Upstream-Downstream Decoupled Architecture

## Upstream Policy Network (Trainable)

- **Backbone:** XLM-RoBERTa (560M parameters).
- **Role:** Emits independent retention probabilities  $\pi_{\theta}(m_i = 1 \mid X)$  for each prompt token.
- Samples binary decision vector  $m \in \{0, 1\}^N$ .

## Downstream Target LM (100% Frozen)

- **Backbone:** Microsoft Phi-4 (14B parameters).
- **Quantization:** 4-bit NormalFloat (NF4).
- **Role:** Evaluates pruned prompt  $X'$  and target sequence  $Y$  without backpropagation.

**Workflow:**  $X \xrightarrow{\text{XLM-R}} m \in \{0, 1\}^N \xrightarrow{\text{Prune}} X' \xrightarrow{\text{Phi-4 (Frozen)}} P(y_t \mid y_{<t}, X')$

# Trajectory-Aware Dense Reward Formulation

- **Sparse Reward Failure:** Metrics like final BLEU or Exact Match assign a single scalar at sequence end, failing to identify *which* dropped prompt token caused the failure.
- **Dense Trajectory Divergence:** We penalize distribution shift across the entire reference answer trajectory ( $t = 1 \dots T$ ):

$$\mathcal{R}_{\text{traj}}(X, m) = -\frac{1}{T} \sum_{t=1}^T D_{\text{KL}}\left(P_{\text{frozen}}(\cdot \mid y_{<t}, X) \parallel P_{\text{frozen}}(\cdot \mid y_{<t}, X')\right) + \beta \cdot \mathcal{R}_{\text{length}}(m)$$

- **Single-Pass Causal Computation:** Evaluated in a single forward pass over target tokens using causal maskingno iterative autoregressive generation required during training.
- **Credit Assignment:** Early tokens ( $t = 1 \dots 4$ , entity heads) vs. late tokens ( $t \geq 10$ , terminal syntax) are individually trackable.

# Policy Gradient Optimization

- Let prompt tokens be  $X = (x_1, \dots, x_N)$ . The policy outputs independent retention Bernoulli probabilities:

$$\pi_{\theta}(m \mid X) = \prod_{i=1}^N \pi_{\theta}(m_i \mid X)$$

- The policy objective maximizes expected trajectory reward and compression incentive:

$$J(\theta) = \mathbb{E}_{m \sim \pi_{\theta}} [\mathcal{R}_{\text{traj}}(X, m)]$$

- REINFORCE Gradient Estimator with Baseline Subtraction:**

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{m \sim \pi_{\theta}} \left[ \sum_{i=1}^N \nabla_{\theta} \log \pi_{\theta}(m_i \mid X) \cdot (\mathcal{R}_{\text{traj}}(X, m) - b(X)) \right]$$

where baseline  $b(X)$  reduces gradient variance across training batches.

# Distributed Scaling via PyTorch DDP All-Reduce

- **Sentence Diversity:** Unlike single-GPU rollouts (TACO-RL), 4-GPU DDP processes 4 distinct context-question pairs concurrently across devices.
- **Independent Actions:** Discrete pruning masks  $m^{(k)}$  are sampled independently on GPU  $k$  and are **never averaged**.
- **NCCL Gradient All-Reduce:** Gradients are synchronized and averaged across all workers:

$$\bar{\nabla}_{\theta} = \frac{1}{4} \sum_{k=1}^4 \nabla_{\theta} \mathcal{L}_k(\theta)$$

## Why This Matters

Synchronizing policy gradients rather than action distributions ensures that the upstream policy generalises across diverse syntactic patterns while preserving discrete token decisions.

# Experimental Setup

- **Dataset:** SQuAD reading comprehension benchmark (passages + context questions).
- **Upstream Editor:** XLM-RoBERTa (560M), learning rate  $1 \times 10^{-5}$ , linear  $\beta = 0.5$  length penalty.
- **Downstream Target:** Microsoft Phi-4 (14B, 4-bit NF4), 100% frozen parameters.
- **Compared Regimes:**
  - ① **Token-0 Baseline:** Sparse supervision targeting only the first answer token (300 steps).
  - ② **Traj RL (1-GPU):** Dense trajectory supervision on single GPU (300 steps).
  - ③ **Traj RL (4-GPU):** Distributed 4-GPU DDP with NCCL All-Reduce (300 steps).
  - ④ **Traj RL Full (87k):** High-budget scaling run (21.9k steps / 87k total sample passes).

# Macro Benchmark Results

| Model / Experiment     | Steps      | Drop Rate     | Downstream F1 | EM (%)       | Step $t = 4$ Drift | Step $t = 10$ Drift |
|------------------------|------------|---------------|---------------|--------------|--------------------|---------------------|
| Token-0 Baseline       | 300        | 74.14%        | 0.5991        | 6.7%         | 1.5340             | 1.1162              |
| Traj RL (1-GPU)        | 300        | 20.77%        | 0.7978        | 46.7%        | 0.4017             | <b>0.0658</b>       |
| <b>Traj RL (4-GPU)</b> | <b>300</b> | <b>46.94%</b> | <b>0.7542</b> | <b>33.3%</b> | <b>0.9309</b>      | <b>0.3560</b>       |
| Traj RL Full (87k)     | 21.9k      | 75.93%        | 0.5772        | 6.7%         | <b>0.2412</b>      | 1.2952              |

- **Token-0 Failure:** Prunes aggressively (74.14%) but degrades F1 and EM due to lack of trajectory feedback.
- **Traj RL 4-GPU Pareto Efficiency:** Achieves a balanced trade-offprunes **46.94% of tokens** while retaining a strong **0.7542 F1** and **33.3% Exact Match**.

# The 87k Run: Policy Saturation & Hallucination Cliff

## What Happened at 87k Steps?

- Training with static  $\beta = 0.5$  length penalty caused the policy to prioritize token deletion over preservation.
- Compression climbed to **75.93% drop**, but Exact Match collapsed to **6.7%**.

## Trajectory Divergence Analysis:

- Early step drift ( $t = 4$ ) remained very low: **0.2412** (entity heads preserved).
- Late step drift ( $t = 10$ ) exploded to **1.2952** (syntactic context destroyed).

## Theoretical Takeaway

Over-pruning removes supporting syntactic framing. While the model correctly initiates generation, the lack of context triggers autoregressive drift and hallucinations.

**Remedy:** Requires dynamic  $\beta$ -annealing or early stopping once Pareto inflection is reached.

# Conclusion & Contributions

- ① **Discrete Prompt Pruning Formulation:** Validated that hard binary masks ( $m \in \{0, 1\}^N$ ) trained via REINFORCE eliminate test-time distribution collapse inherent to soft masks.
- ② **Dense Trajectory Supervision:** Replaced uninformative sequence-level rewards with step-wise causal  $D_{\text{KL}}$  trajectory loss, enabling fine-grained credit assignment.
- ③ **Distributed Exploration:** Demonstrated that 4-GPU DDP gradient All-Reduce discovers robust pruning policies, achieving **46.94% compression** with **0.7542 F1**.
- ④ **Identified Policy Saturation Dynamics:** Documented the 87k hallucination cliff, establishing the necessity of dynamic length-penalty scheduling in RL-based compression.

Thank you. Questions?